

Atividade Avaliativa com Bônus — Polimorfismo em C

Esta atividade tem como objetivo consolidar o estudo de polimorfismo em C#, exigindo que cada aluno analise um problema, abstraia corretamente os elementos do domínio, identifique propriedades e comportamentos comuns, crie uma estrutura hierárquica coerente e aplique polimorfismo na execução do sistema.

Cada aluno deverá implementar o problema individual recebido, criando uma solução em C# com separação mínima entre domínio e execução. A implementação deverá conter uma estrutura orientada a objetos consistente, com classe base abstrata, classes concretas especializadas, propriedades comuns, propriedades específicas, métodos abstratos e/ou virtuais, sobrescrita de métodos com `override`, lista polimórfica e teste da execução no `Program.cs`.

A entrega deve ser realizada até o final da aula por meio de um repositório no GitHub. Após finalizar a implementação, o aluno deverá chamar o professor para apresentar o funcionamento do projeto. A entrega funcional, organizada e coerente com os princípios de polimorfismo poderá gerar até 0,5 ponto de bônus.

O aluno que desejar concorrer a mais 0,5 ponto de bônus deverá gravar, até sexta-feira, um vídeo em formato de apresentação ou seminário. Esse vídeo deverá demonstrar os princípios relacionados ao polimorfismo no problema implementado, explicando a abstração criada, a hierarquia de classes, os métodos comuns, os comportamentos sobrescritos, o uso da lista polimórfica e a execução do sistema.

Requisitos obrigatórios para todos os problemas

A solução deve conter, no mínimo:

1. Uma solução C# com dois projetos:
2. um projeto de biblioteca de classes para o domínio;
3. um projeto console para execução e teste.
4. Uma classe base abstrata representando a abstração principal do domínio.
5. Pelo menos quatro classes concretas derivadas da classe base.
6. Propriedades comuns na classe abstrata, com validações no construtor.
7. Pelo menos uma propriedade específica em cada classe concreta.
8. Pelo menos um método abstrato que obrigue cada classe concreta a implementar seu próprio comportamento.
9. Pelo menos um método concreto na classe abstrata, reutilizado pelas classes derivadas.

10. Uma classe de serviço responsável por armazenar os objetos em uma coleção polimórfica.
 11. Uso de `List<T>` com o tipo da classe abstrata.
 12. Execução no `Program.cs` criando objetos concretos diferentes, adicionando-os à coleção e chamando os métodos polimórficos.
 13. Ausência de `if`, `else if` ou `switch` para decidir comportamento com base no tipo concreto do objeto.
 14. Organização clara de pastas, classes e responsabilidades.
 15. Comentários ou documentação breve explicando a abstração escolhida e a lógica da hierarquia.
-

Enunciados individuais

1. Ana Vitoria Paula Apolinario — Sistema de Eventos Acadêmicos

Você deverá implementar um sistema para registrar diferentes tipos de eventos acadêmicos promovidos por uma instituição de ensino. O sistema deve permitir cadastrar eventos como palestra, minicurso, oficina prática e mesa-redonda. Todos os eventos possuem título, carga horária, responsável e quantidade estimada de participantes, mas cada tipo de evento possui regras próprias para gerar sua descrição, calcular seu impacto formativo e indicar o tipo de participação esperada do estudante.

Sua tarefa é abstrair o conceito geral de evento acadêmico, identificar quais propriedades e comportamentos são comuns a todos os eventos e criar uma hierarquia de classes que represente adequadamente as especializações. O sistema deve possuir uma classe abstrata para o evento acadêmico e classes concretas para os diferentes tipos de evento.

No `Program.cs`, crie uma lista polimórfica com diferentes eventos e exiba, para cada um, sua descrição completa, seu impacto formativo e sua orientação de participação. O código principal não deve verificar se o evento é palestra, minicurso, oficina ou mesa-redonda; cada classe concreta deve saber gerar seu próprio comportamento.

2. Cler Elis Andrade — Sistema de Gestão de Processos Seletivos

Você deverá implementar um sistema para gerenciar diferentes tipos de etapas em um processo seletivo acadêmico ou institucional. O sistema deve considerar, no mínimo, as seguintes etapas: análise documental, prova técnica, entrevista e avaliação de currículo. Todas as etapas possuem nome, candidato avaliado, peso na seleção, nota obtida e avaliador responsável, mas cada tipo de etapa possui uma forma própria de gerar parecer, calcular pontuação ponderada e indicar pendências ou recomendações.

Sua tarefa é abstrair o conceito geral de etapa de processo seletivo, identificando quais propriedades e comportamentos pertencem a todas as etapas e quais características devem ficar apenas nas classes

concretas. A solução deve conter uma classe base abstrata, por exemplo `EtapaProcessoSeletivo`, e classes derivadas para representar cada tipo específico de etapa.

A classe abstrata deve possuir propriedades comuns, validações no construtor e pelo menos um método concreto reutilizável pelas classes derivadas. Também deve possuir métodos abstratos que obriguem cada etapa concreta a implementar sua própria forma de gerar parecer, calcular pontuação ponderada e indicar recomendações.

Cada classe concreta deve possuir pelo menos uma propriedade específica. Por exemplo, a análise documental pode possuir quantidade de documentos avaliados; a prova técnica pode possuir linguagem ou ferramenta utilizada; a entrevista pode possuir modalidade; e a avaliação de currículo pode possuir quantidade de experiências consideradas.

No `Program.cs`, crie diferentes etapas do processo seletivo, armazene todas em uma lista polimórfica e gere um relatório final do candidato. Esse relatório deve exibir, para cada etapa, a descrição base, o parecer específico, a pontuação ponderada e as recomendações. O código principal não deve usar `if`, `else if` ou `switch` para descobrir qual é o tipo da etapa. Cada classe concreta deve ser responsável por executar seu próprio comportamento.

Ao final, demonstre também o cálculo da pontuação total do processo seletivo, percorrendo a lista polimórfica e somando as pontuações ponderadas retornadas pelos próprios objetos.

3. Edio Nienov Neto — Sistema de Manutenção de Equipamentos de Laboratório

Você deverá implementar um sistema para controle de manutenções em equipamentos de laboratório de informática. O sistema deve lidar com diferentes tipos de manutenção, como manutenção preventiva, manutenção corretiva, atualização de software e substituição de componente. Todas as manutenções possuem código de registro, equipamento atendido, técnico responsável e tempo estimado, mas cada tipo possui forma própria de gerar relatório, calcular prioridade e indicar risco operacional.

Sua tarefa é criar uma abstração geral para manutenção de equipamento e modelar as classes concretas conforme suas diferenças. A classe abstrata deve concentrar os dados comuns e obrigar as classes derivadas a implementar os comportamentos específicos.

No `Program.cs`, crie diferentes objetos de manutenção, armazene todos em uma lista polimórfica e percorra a lista para exibir o relatório de cada manutenção. O sistema deve demonstrar que objetos diferentes podem ser tratados pelo mesmo tipo abstrato, sem uso de condicionais por tipo.

4. Enzo Schmidt Sossella — Sistema de Avaliação de Entregas de Projeto

Você deverá implementar um sistema para avaliação de diferentes entregas de um projeto acadêmico. O sistema deve considerar entregas como documento de requisitos, protótipo de interface, implementação parcial e apresentação técnica. Todas as entregas possuem título, peso, nota obtida e

responsável, mas cada entrega possui critérios próprios para gerar feedback, calcular pontuação ponderada e indicar melhorias.

A solução deve conter uma abstração geral para entrega avaliativa e classes concretas para cada tipo de entrega. Cada classe concreta deve implementar de forma própria o comportamento de gerar feedback e calcular a pontuação final, considerando suas regras específicas.

No `Program.cs`, crie uma coleção polimórfica de entregas e exiba um relatório geral com feedbacks individuais, pontuação ponderada de cada entrega e total acumulado. O relatório deve depender apenas da abstração, não das classes concretas.

5. Ernest Beuter Grob Machado — Sistema de Atendimentos em Suporte Técnico

Você deverá implementar um sistema para registrar atendimentos de suporte técnico realizados por uma equipe de tecnologia. O sistema deve trabalhar com chamados de acesso, chamados de rede, chamados de hardware e chamados de software. Todos os chamados possuem solicitante, descrição, data de abertura e nível de urgência, mas cada tipo possui forma própria de estimar tempo de atendimento, gerar diagnóstico e sugerir solução.

Crie uma classe abstrata que represente o conceito geral de chamado técnico. Em seguida, crie classes concretas para os tipos de chamado. Cada classe concreta deve possuir pelo menos uma propriedade específica e sobrescrever os comportamentos definidos na abstração.

No `Program.cs`, crie diferentes chamados, armazene-os em uma lista polimórfica e gere um painel de atendimento. O painel deve mostrar o diagnóstico, a solução proposta e o tempo estimado, sem que o código principal pergunte qual é o tipo do chamado.

6. Evandro Grichok Pereira Dos Santos — Sistema de Gestão de Reservas de Espaços

Você deverá implementar um sistema para gerenciar reservas de espaços institucionais. O sistema deve lidar com reserva de laboratório, sala de aula, auditório e sala de reunião. Todas as reservas possuem solicitante, data, horário inicial, duração e finalidade, mas cada tipo de espaço possui regras próprias para validar sua utilização, gerar instruções de uso e calcular prioridade de aprovação.

A solução deve conter uma classe abstrata que represente uma reserva de espaço. As classes concretas devem representar os diferentes tipos de reserva e implementar seus próprios comportamentos. O sistema deve ser capaz de tratar todas as reservas de forma uniforme por meio da abstração.

No `Program.cs`, crie uma lista polimórfica de reservas e exiba para cada uma: descrição da reserva, instruções específicas de uso e prioridade de aprovação. Não utilize `switch` ou `if` para decidir o comportamento de cada tipo de espaço.

7. Fabio Rafael Santana — Sistema de Processamento de Inscrições em Atividades

Você deverá implementar um sistema para processar inscrições de estudantes em diferentes atividades acadêmicas. O sistema deve considerar inscrição em monitoria, projeto de extensão, iniciação científica e curso complementar. Todas as inscrições possuem estudante, data de inscrição, carga horária pretendida e status inicial, mas cada tipo possui critérios próprios para gerar análise, calcular relevância acadêmica e indicar documentação necessária.

Crie uma classe abstrata para inscrição acadêmica e classes concretas para cada tipo de inscrição. Os comportamentos variáveis devem ser definidos como métodos abstratos ou virtuais e implementados pelas classes concretas.

No `Program.cs`, crie diferentes inscrições e processe todas por meio de uma lista polimórfica. O resultado deve exibir a análise da inscrição, a documentação necessária e a relevância acadêmica calculada para cada caso.

8. Felipe Antonio Cecconi — Sistema de Gerenciamento de Contratos de Serviço

Você deverá implementar um sistema para gerenciar contratos de serviço de uma empresa. O sistema deve trabalhar com contrato de suporte técnico, contrato de consultoria, contrato de manutenção e contrato de treinamento. Todos os contratos possuem cliente, valor base, prazo em meses e responsável, mas cada tipo possui forma própria de calcular valor final, gerar resumo contratual e indicar obrigações principais.

A solução deve conter uma classe abstrata para contrato de serviço e classes concretas especializadas. Cada contrato concreto deve possuir pelo menos uma propriedade exclusiva, como quantidade de horas, nível de suporte, número de visitas ou quantidade de turmas.

No `Program.cs`, crie uma coleção polimórfica de contratos e gere um relatório exibindo resumo, valor final e obrigações de cada contrato. O código consumidor deve trabalhar apenas com a abstração contrato, sem conhecer as classes concretas.

9. Felipe Eduardo Santana — Sistema de Controle de Publicações Acadêmicas

Você deverá implementar um sistema para organizar diferentes tipos de publicações acadêmicas. O sistema deve lidar com artigo científico, resumo expandido, capítulo de livro e relatório técnico. Todas as publicações possuem título, autores, ano e área temática, mas cada tipo possui regras próprias para gerar referência, calcular relevância acadêmica e indicar veículo de publicação.

Crie uma classe abstrata para publicação acadêmica e classes concretas para os diferentes tipos de publicação. Os métodos de gerar referência e calcular relevância devem ser polimórficos.

No `Program.cs`, crie diferentes publicações, armazene todas em uma lista polimórfica e gere uma listagem de referências com relevância calculada. O sistema deve demonstrar claramente que o mesmo método chamado sobre a abstração executa comportamentos diferentes conforme o objeto real.

10. Gabriel Kaua Canalle — Sistema de Operações Bancárias

Você deverá implementar um sistema para registrar diferentes operações bancárias em uma conta. O sistema deve considerar depósito, saque, transferência e pagamento de boleto. Todas as operações possuem valor, data, descrição e identificador, mas cada tipo possui regra própria para calcular tarifa, gerar comprovante e indicar impacto no saldo.

A solução deve conter uma classe abstrata para operação bancária e classes concretas para cada tipo de operação. Cada classe concreta deve implementar seu próprio cálculo de tarifa e sua própria geração de comprovante.

No `Program.cs`, crie uma lista polimórfica de operações e gere um extrato detalhado. O extrato deve exibir o comprovante, a tarifa e o impacto no saldo de cada operação. O código principal não deve usar condicionais para descobrir se a operação é depósito, saque, transferência ou boleto.

11. Gabriela Oneda Correa — Sistema de Avaliação de Conteúdos Educacionais

Você deverá implementar um sistema para avaliar diferentes conteúdos educacionais disponibilizados em uma plataforma. O sistema deve lidar com videoaula, apostila, quiz e atividade prática. Todos os conteúdos possuem título, duração estimada, tema e pontuação máxima, mas cada tipo possui forma própria de calcular engajamento, gerar orientação ao estudante e indicar critério de conclusão.

Crie uma classe abstrata para conteúdo educacional e classes concretas para cada tipo de conteúdo. Cada classe concreta deve implementar os comportamentos específicos, evitando que o código externo precise saber qual é o tipo do conteúdo.

No `Program.cs`, crie uma lista polimórfica de conteúdos e exiba um painel com orientação, critério de conclusão e engajamento estimado para cada conteúdo. O painel deve ser gerado por chamadas polimórficas.

12. Guilherme Pastore Zornitta — Sistema de Controle de Fretes

Você deverá implementar um sistema para calcular diferentes tipos de frete em uma transportadora. O sistema deve considerar frete rodoviário, frete aéreo, frete expresso e frete refrigerado. Todos os fretes possuem origem, destino, peso da carga e valor base, mas cada tipo possui regra própria para calcular custo final, estimar prazo e gerar instruções de transporte.

A solução deve conter uma classe abstrata para frete e classes concretas para cada modalidade. Cada tipo de frete deve possuir pelo menos uma propriedade específica, como taxa de urgência, controle de temperatura, distância aérea ou pedágios.

No `Program.cs`, crie diferentes fretes, armazene todos em uma lista polimórfica e gere uma simulação de envio. A simulação deve exibir custo final, prazo estimado e instruções específicas, sem utilizar condicionais baseadas no tipo de frete.

13. Guilherme Volpato Nestor — Sistema de Gestão de Tarefas de Projeto

Você deverá implementar um sistema para gerenciar tarefas de um projeto de software. O sistema deve considerar tarefa de análise, tarefa de implementação, tarefa de teste e tarefa de documentação. Todas as tarefas possuem título, responsável, prazo e complexidade, mas cada tipo possui forma própria de calcular esforço, gerar instrução de execução e indicar critério de aceite.

Crie uma classe abstrata para tarefa de projeto e classes concretas para as especializações. Cada classe concreta deve implementar seu próprio cálculo de esforço e sua própria descrição de critério de aceite.

No `Program.cs`, crie uma lista polimórfica de tarefas e gere um quadro de acompanhamento. O quadro deve exibir, para cada tarefa, esforço estimado, instrução de execução e critério de aceite. O serviço responsável pelo quadro deve trabalhar apenas com a abstração.

14. Gustavo Andre Kogut — Sistema de Gestão de Planos de Assinatura

Você deverá implementar um sistema para gerenciar planos de assinatura de uma plataforma digital. O sistema deve possuir plano básico, plano profissional, plano educacional e plano empresarial. Todos os planos possuem nome, valor mensal, limite de usuários e descrição, mas cada plano possui regra própria para calcular valor anual, gerar benefícios e indicar restrições.

A solução deve conter uma classe abstrata para plano de assinatura e classes concretas para cada tipo de plano. Os métodos de cálculo anual, geração de benefícios e descrição de restrições devem ser tratados de forma polimórfica.

No `Program.cs`, crie diferentes planos, adicione-os a uma lista polimórfica e gere uma tabela textual de comparação. A comparação deve ser feita por meio da abstração, sem que o código principal teste o tipo concreto de plano.

15. Henrique Goncalves Moresco — Sistema de Controle de Produção Rural

Você deverá implementar um sistema para registrar atividades de produção rural. O sistema deve considerar plantio, irrigação, adubação e colheita. Todas as atividades possuem área, data, responsável e cultura agrícola, mas cada tipo possui regras próprias para calcular custo estimado, gerar recomendação técnica e indicar risco operacional.

Crie uma classe abstrata para atividade rural e classes concretas para cada tipo de atividade. Cada classe concreta deve possuir propriedades específicas, como tipo de semente, volume de água, tipo de adubo ou produtividade estimada.

No `Program.cs`, crie uma lista polimórfica de atividades e gere um relatório técnico da propriedade. O relatório deve exibir custo, recomendação e risco de cada atividade por meio de chamadas polimórficas.

16. Iago Monteiro Lima — Sistema de Gestão de Ocorrências Escolares

Você deverá implementar um sistema para registrar diferentes ocorrências em ambiente escolar. O sistema deve considerar ocorrência disciplinar, ocorrência pedagógica, ocorrência de saúde e ocorrência administrativa. Todas as ocorrências possuem estudante envolvido, data, descrição e responsável pelo registro, mas cada tipo possui forma própria de gerar encaminhamento, calcular nível de atenção e indicar setor responsável.

A solução deve conter uma classe abstrata para ocorrência escolar e classes concretas para cada tipo. Cada ocorrência concreta deve implementar os comportamentos específicos, mantendo as regras dentro das próprias classes.

No `Program.cs`, crie diferentes ocorrências, armazene-as em uma lista polimórfica e gere um painel de encaminhamento. O painel deve mostrar setor responsável, nível de atenção e orientação gerada para cada ocorrência.

17. Joao Vitor Brandao Redigolo — Sistema de Controle de Pedidos em Restaurante

Você deverá implementar um sistema para controlar diferentes tipos de pedidos em um restaurante. O sistema deve considerar pedido presencial, pedido para retirada, pedido delivery e pedido corporativo. Todos os pedidos possuem cliente, valor dos itens, data e observações, mas cada tipo possui regra própria para calcular taxa adicional, gerar instrução de preparo e definir prioridade.

Crie uma classe abstrata para pedido e classes concretas para as quatro modalidades. Cada classe concreta deve possuir pelo menos uma propriedade específica, como endereço de entrega, horário de retirada, número da mesa ou quantidade de pessoas.

No `Program.cs`, crie uma lista polimórfica de pedidos e gere um painel de cozinha. O painel deve exibir prioridade, taxa adicional e instrução de preparo sem usar `if`, `else if` ou `switch` para decidir o comportamento.

18. Kali Gomes De Freitas Costa — Sistema de Gestão de Consultas em Clínica

Você deverá implementar um sistema para registrar diferentes atendimentos em uma clínica. O sistema deve considerar consulta médica, exame laboratorial, sessão de fisioterapia e retorno clínico. Todos os atendimentos possuem paciente, profissional responsável, data e duração prevista, mas cada tipo possui forma própria de gerar resumo, calcular custo e indicar preparo necessário.

A solução deve conter uma classe abstrata para atendimento clínico e classes concretas para os diferentes atendimentos. Cada classe concreta deve implementar seu próprio resumo e sua própria regra de cálculo de custo.

No `Program.cs`, crie uma lista polimórfica de atendimentos e gere uma agenda detalhada. A agenda deve apresentar resumo, custo e preparo de cada atendimento, utilizando apenas a abstração atendimento clínico.

19. Leonardo Hammes — Sistema de Gestão de Licenças de Software

Você deverá implementar um sistema para controlar diferentes licenças de software em uma organização. O sistema deve considerar licença individual, licença por volume, licença educacional e licença temporária. Todas as licenças possuem nome do software, titular, data de início e valor base, mas cada tipo possui regra própria para calcular validade, gerar restrições de uso e calcular custo final.

Crie uma classe abstrata para licença de software e classes concretas para cada tipo. Cada classe concreta deve possuir propriedades específicas, como quantidade de usuários, percentual de desconto, data de expiração ou vínculo institucional.

No `Program.cs`, crie uma lista polimórfica de licenças e gere um relatório de auditoria. O relatório deve exibir validade, restrições e custo final de cada licença sem depender de condicionais por tipo.

20. Luis Felipe Machado Dutra — Sistema de Gestão de Incidentes de Segurança da Informação

Você deverá implementar um sistema para registrar e tratar diferentes tipos de incidentes de segurança da informação em uma organização. O sistema deve considerar, no mínimo, os seguintes incidentes: tentativa de phishing, vazamento de dados, acesso não autorizado e malware detectado. Todos os incidentes possuem identificador, data de registro, responsável pela análise, sistema afetado e nível inicial de severidade, mas cada tipo de incidente possui uma forma própria de calcular risco, gerar plano de resposta e indicar medidas preventivas.

Sua tarefa é abstrair o conceito geral de incidente de segurança, identificando as propriedades e comportamentos comuns a todos os incidentes e separando adequadamente aquilo que pertence apenas a cada tipo específico. A solução deve conter uma classe base abstrata, por exemplo `IncidenteSeguranca`, e classes concretas para cada tipo de incidente.

A classe abstrata deve concentrar os dados comuns, aplicar validações no construtor e fornecer pelo menos um método concreto para gerar uma descrição base do incidente. Além disso, deve declarar métodos abstratos para os comportamentos que variam entre os tipos, como `CalcularRisco()`, `GerarPlanoResposta()` e `IndicarMedidasPreventivas()`.

Cada classe concreta deve possuir pelo menos uma propriedade específica. Por exemplo, o phishing pode possuir canal de recebimento; o vazamento de dados pode possuir quantidade estimada de registros afetados; o acesso não autorizado pode possuir origem do acesso; e o malware pode possuir nome ou categoria da ameaça detectada.

No `Program.cs`, crie uma lista polimórfica de incidentes de segurança e adicione objetos de todos os tipos concretos. Em seguida, percorra essa lista para gerar um relatório de resposta a incidentes, exibindo a descrição base, o risco calculado, o plano de resposta e as medidas preventivas de cada incidente.

O código consumidor deve trabalhar apenas com a abstração `IncidenteSeguranca`. Não será aceito um código que utilize condicionais para decidir o comportamento com base no tipo concreto do incidente. O objetivo é demonstrar que cada incidente sabe calcular seu próprio risco e gerar sua própria resposta, enquanto o `Program.cs` apenas organiza e executa o fluxo.

21. Maisa Vendrame Kuhne — Sistema de Gestão de Campanhas de Marketing

Você deverá implementar um sistema para organizar diferentes campanhas de marketing. O sistema deve considerar campanha em rede social, campanha por e-mail, campanha presencial e campanha com influenciadores. Todas as campanhas possuem nome, orçamento, público-alvo e período de execução, mas cada tipo possui forma própria de calcular alcance estimado, gerar estratégia e indicar métrica principal.

A solução deve conter uma classe abstrata para campanha e classes concretas para cada modalidade. Cada campanha concreta deve implementar seus próprios comportamentos, especialmente cálculo de alcance e geração de estratégia.

No `Program.cs`, crie uma lista polimórfica de campanhas e gere um plano consolidado. O plano deve exibir estratégia, alcance estimado e métrica principal de cada campanha.

22. Maria Julia Becker — Sistema de Controle de Treinamentos Corporativos

Você deverá implementar um sistema para gerenciar treinamentos corporativos. O sistema deve considerar treinamento técnico, treinamento comportamental, treinamento obrigatório e treinamento de integração. Todos os treinamentos possuem título, instrutor, carga horária e número de participantes, mas cada tipo possui regra própria para gerar objetivo, calcular custo e indicar forma de avaliação.

Crie uma classe abstrata para treinamento e classes concretas para cada tipo. Cada treinamento concreto deve implementar os métodos de gerar objetivo, calcular custo e indicar avaliação.

No `Program.cs`, crie diferentes treinamentos, armazene todos em uma lista polimórfica e gere um cronograma de capacitação. O cronograma deve depender apenas da abstração treinamento.

23. Mateus Borges Tonin — Sistema de Gestão de Apólices de Seguro

Você deverá implementar um sistema para gerenciar diferentes apólices de seguro. O sistema deve considerar seguro residencial, seguro automotivo, seguro de vida e seguro empresarial. Todas as apólices possuem segurado, valor segurado, vigência e corretor responsável, mas cada tipo possui forma própria de calcular prêmio, gerar cobertura e indicar exclusões.

A solução deve conter uma classe abstrata para apólice de seguro e classes concretas para cada tipo. Cada classe concreta deve possuir propriedades específicas, como modelo do veículo, idade do segurado, endereço do imóvel ou porte da empresa.

No `Program.cs`, crie uma lista polimórfica de apólices e gere um relatório de seguros. O relatório deve exibir prêmio, cobertura e exclusões por meio de chamadas polimórficas.

24. Moises Grassi — Sistema de Gestão de Chamados de Infraestrutura

Você deverá implementar um sistema para registrar chamados de infraestrutura predial. O sistema deve considerar chamado elétrico, chamado hidráulico, chamado de climatização e chamado de mobiliário. Todos os chamados possuem local, solicitante, descrição e data de abertura, mas cada tipo possui forma própria de calcular prioridade, gerar equipe responsável e estimar tempo de resolução.

Crie uma classe abstrata para chamado de infraestrutura e classes concretas para cada categoria. Cada classe concreta deve implementar seus comportamentos específicos e possuir propriedades próprias.

No `Program.cs`, crie uma lista polimórfica de chamados e gere uma fila de atendimento. A fila deve mostrar equipe responsável, prioridade e tempo estimado, sem utilizar condicionais para identificar o tipo do chamado.

25. Nilseo Cassol — Sistema de Controle de Produtos em Estoque

Você deverá implementar um sistema para controlar diferentes produtos em estoque. O sistema deve considerar produto perecível, produto eletrônico, produto de limpeza e produto controlado. Todos os produtos possuem nome, código, quantidade e valor unitário, mas cada tipo possui regras próprias para calcular risco de estoque, gerar instrução de armazenamento e calcular valor total ajustado.

A solução deve conter uma classe abstrata para produto e classes concretas para cada tipo. Cada classe concreta deve possuir pelo menos uma propriedade específica, como data de validade, garantia, composição química ou exigência de autorização.

No `Program.cs`, crie uma lista polimórfica de produtos e gere um relatório de estoque. O relatório deve exibir valor ajustado, risco e instrução de armazenamento para cada produto.

26. Pablo Augusto Weber — Sistema de Gestão de Rotas de Entrega

Você deverá implementar um sistema para planejar rotas de entrega. O sistema deve considerar rota urbana, rota rural, rota interestadual e rota expressa. Todas as rotas possuem origem, destino, distância e motorista responsável, mas cada tipo possui regra própria para calcular custo, estimar tempo e gerar recomendações de execução.

Crie uma classe abstrata para rota de entrega e classes concretas para cada tipo de rota. Cada classe concreta deve implementar comportamentos específicos e possuir propriedades exclusivas, como quantidade de paradas, condição da estrada, pedágios ou taxa de urgência.

No `Program.cs`, crie uma lista polimórfica de rotas e gere um plano de distribuição. O plano deve exibir custo, tempo e recomendação para cada rota usando apenas a abstração.

27. Rafael Debarba — Sistema de Gestão de Projetos de Pesquisa

Você deverá implementar um sistema para registrar diferentes projetos de pesquisa. O sistema deve considerar projeto de iniciação científica, projeto de extensão tecnológica, projeto de inovação e projeto de desenvolvimento experimental. Todos os projetos possuem título, coordenador, área e duração em meses, mas cada tipo possui forma própria de gerar objetivo, calcular impacto e indicar produto esperado.

A solução deve conter uma classe abstrata para projeto acadêmico e classes concretas para cada tipo. Cada projeto concreto deve implementar seus próprios métodos para objetivo, impacto e produto esperado.

No `Program.cs`, crie uma lista polimórfica de projetos e gere um portfólio institucional. O portfólio deve tratar todos os projetos pela abstração, sem conhecer diretamente as classes específicas.

28. Thiago Correa De Lima — Sistema de Gestão de Missões Educacionais

Você deverá implementar um sistema para organizar missões educacionais em uma plataforma de aprendizagem. O sistema deve considerar missão de leitura, missão de prática, missão de quiz e missão de projeto. Todas as missões possuem título, tema, XP máximo e prazo, mas cada tipo possui regra própria para calcular progresso, gerar orientação ao estudante e definir critério de conclusão.

Crie uma classe abstrata para missão educacional e classes concretas para cada tipo de missão. Cada classe concreta deve possuir propriedades específicas, como número de páginas, quantidade de exercícios, quantidade de questões ou artefato final esperado.

No `Program.cs`, crie uma lista polimórfica de missões e gere um painel de acompanhamento. O painel deve exibir progresso, orientação e critério de conclusão para cada missão. O código principal deve trabalhar somente com a abstração missão educacional.

Orientação final aos alunos

Ao implementar o projeto, não basta criar classes com nomes diferentes. É necessário demonstrar que houve uma análise do domínio. Antes de codificar, identifique:

- qual é a ideia geral do problema;
- quais dados pertencem a todos os objetos;
- quais dados pertencem apenas a alguns tipos específicos;
- qual comportamento todos devem ter;
- qual comportamento muda conforme o tipo concreto;
- por que a classe base deve ser abstrata;
- como a lista polimórfica permite tratar objetos diferentes com o mesmo código.

A implementação será avaliada pela coerência da modelagem, uso correto de herança e polimorfismo, organização da solução, clareza das responsabilidades, ausência de condicionais por tipo e funcionamento demonstrado no `Program.cs`.

Para o vídeo opcional, o aluno deverá apresentar, com clareza:

- o problema recebido;
- a abstração principal escolhida;
- as propriedades comuns da classe abstrata;
- as propriedades específicas das classes concretas;
- os métodos abstratos e/ou virtuais definidos;
- os métodos sobrescritos com `override`;
- o uso da lista polimórfica;
- a execução no `Program.cs`;
- por que a solução evita condicionais baseadas em tipo;
- como o polimorfismo aparece no projeto implementado.